Assignment No. 03

Comsats University Islamabad, Sahiwal Campus

Submitted To:

SIR ALI SHER KASHIF

Submitted By:

Ahmad Husnain

Roll No:

SP25-BSE-063

Course:

DATA STRUCTURE

Q#1: University Student Record System

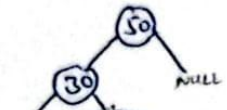
Q NO 1 University Student Record System.

Task: Construct BST

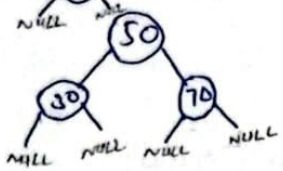
50, 30, 70, 20, 40, 60, 80

i) (insertion) [50]

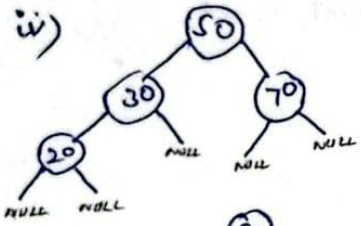
ii) " [50, 30]



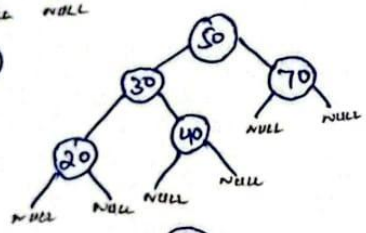
iii) " [50, 30, 70]



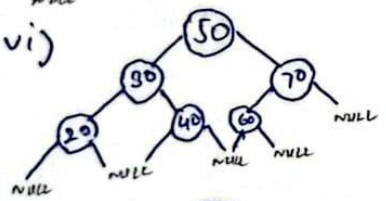
iv) " [50, 30, 70, 20]



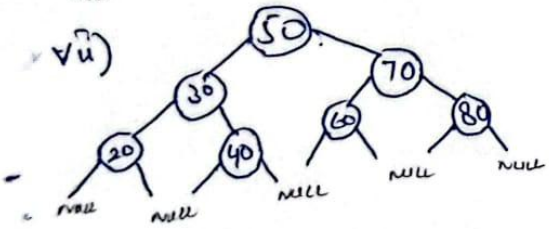
v) " [50, 30, 70, 20, 40]



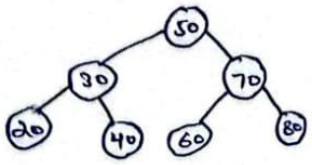
vi) " [50, 30, 70, 20, 40, 60]



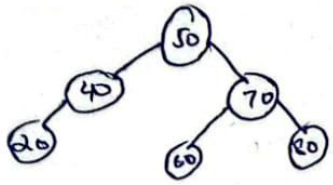
vii) " [50, 30, 70, 20, 40, 60, 80]



•) Delete '30' from tree (BST)

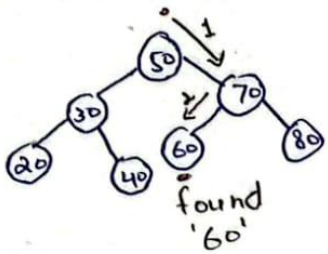


After Deletion



Node 30 removed.

•) Search 60 from BST



$50 \xrightarrow{R} 70 \xrightarrow{L} \boxed{60}$

Movement

Why BST Suitable

- Fast searching
- Fast insertion
- Dynamic Structure
- sorted data store
- Complexity = $O(\log n)$

Code:

```
#include <iostream>
#include <vector>
using namespace std;
struct Node
{
    int data;
    Node* left;
    Node* right;
    Node(int val){
        data  = val;
        left = right = NULL;
    }
};

Node* insert(Node* &root,int val){
    if(root == NULL){
        root = new Node(val);
        return root;
    }
    if(root->data > val)
        root->left = insert(root->left,val);
    else
        root->right = insert(root->right,val);
    return root;
}

bool search(Node* root,int key){
    if( root == NULL)
    return false;
```

```
    if (root->data == key)
    return true;
    if(root->data>key)
    return search(root->left,key);
    else
    return search(root->right,key);
}

Node* minVal(Node* root){
    while(root->left!=NULL){
        root = root->left;
    }
    return root;
}
Node* deleteNode(Node* root,int key){
    if(root==NULL) return NULL;

    if(key < root->data){
        root->left = deleteNode(root->left,key);
    }
    else if (key> root->data){
        root->right = deleteNode(root->right,key);
    }
    else{
        if(root->left == NULL && root->right == NULL){
            delete root;
            return NULL;
        }
        else if (root->left == NULL){
            Node* temp = root->right;
```

```
            delete root;
            return temp;
        }
        else if (root->right == NULL){
            Node* temp = root->left;
            delete root;
            return temp;
        }

        Node* temp = minVal(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right,temp->data);
    }
return root;
}
void InOrder(Node* root){
    if(root==NULL)
    return;
    InOrder(root->left);
    cout<< root->data << " ";
    InOrder(root->right);
}
int main(){
    Node* root = NULL;
    int arr[] = {50,30,70,20,40,60,80};

    for(int x : arr)
        root = insert(root,x);
    cout << "Tree after insertion: ";
    InOrder(root);
```

```
    cout << endl;
    root = deleteNode(root,30);
    if(search(root,60))
        cout<<"Value Found\n";
    else
        cout<<"Value Not Found\n";
    cout << "Tree after deletion: ";
    InOrder(root);
    cout << endl;
    return 0;
}
```

```
PS E:\DSA\Trees> cd "e:\DSA\Trees\" ; if ($?) {
Tree after insertion: 20 30 40 50 60 70 80
Value Found
Tree after deletion: 20 40 50 60 70 80
PS E:\DSA\Trees>
```

Q#2:

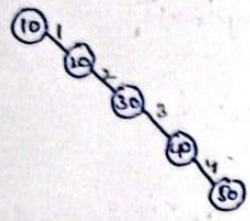
AVL improves efficiency of searching over BST.

BST	AVL	
O(n)	O(log(n))	Time Complexity
No	Yes	Balanced
NO	yes	Fast Ranking

Q_2^{NO} E-commerce Product Ranking

Insertion: 10, 20, 30, 40, 50

BST:

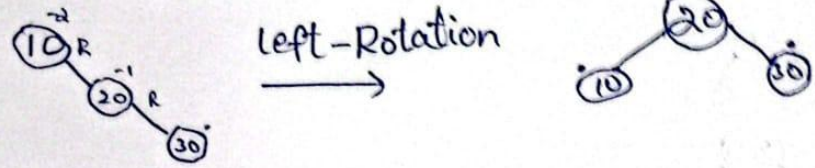


Why becomes Unbalanced:

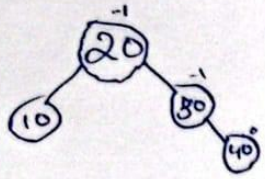
- All nodes inserted on the right side
- Becomes liked Linked-list
- Time complexity: $O(n)$

AVL - Tree

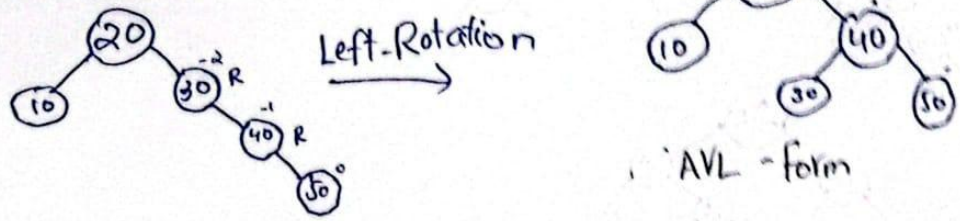
i)



ii)



iii)



AVL - form

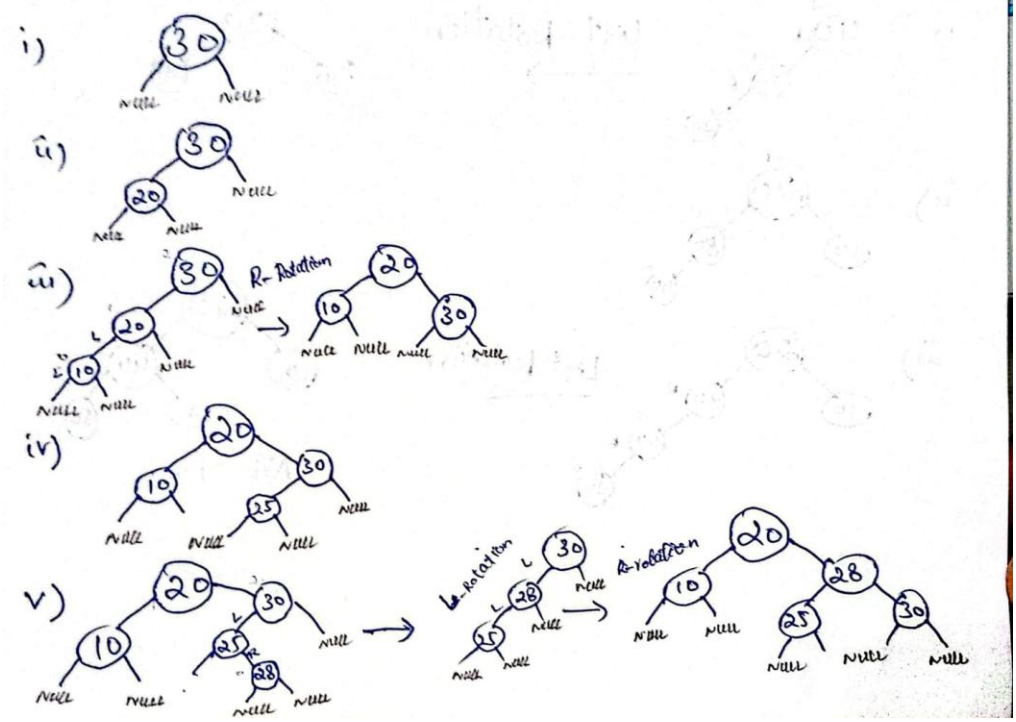
Question#3:

Q3 NO Hospital Emergency AVL Tree

Sequence:

30, 20, 10, 25, 28

Insert AVL:



AVL ensure logarithmic Time:

Height remains small.

Search / insert / delete: $O(\log n)$

improves System over BST.

Code:

```
#include <iostream>
#include <algorithm>
using namespace std;

struct Node{
    int data;
    Node* left;
    Node* right;
    int height;
    Node(int val){
        data = val;
        left = right = NULL;
        height = 1;
    }
};
int getHeight(Node* root){
    if(root==NULL){
        return 0;}
    return root->height;
}
int getBalance(Node* root){
    if(root==NULL){
        return 0;
    }
    return getHeight(root->left) - getHeight(root->right);
}
void InOrder(Node* root){
    if(root==NULL)
    return;
    InOrder(root->left);
```

```
    cout<< root->data << " ";
    InOrder(root->right);
}
Node* RightRotate(Node* y){
    Node* x = y->left;
    Node* T2 = x->right;
    x->right = y;
    y->left = T2;
    return x;
}
Node* LeftRotate(Node* x){
    Node* y = x->right;
    Node* T2 = y->left;
    y->left = x;
    x->right = T2;

    return y;
}

Node* insertNode(Node* root,int key){
    if(root==NULL){
        return new Node(key);
    }

    if(key < root->data){
        root->left = insertNode(root->left,key);
    }
    else if(key > root->data){
        root->right = insertNode(root->right,key);
    }
```

```
    else{
        return root;
    }
    root->height = 1 + max(getHeight(root->left),getHeight(root->right));
    int balance = getHeight(root->left) - getHeight(root->right);
    if(balance >1 && key < root->left->data){
        return RightRotate(root);
    }
     if(balance < 1 && key > root->right->data){
        return LeftRotate(root);
    }
     if(balance > 1 && key > root->left->data){
        root = LeftRotate(root->left);
        return RightRotate(root);
    }
     if(balance < -1 && key < root->right->data ){
        root = RightRotate(root->right);
        return LeftRotate(root);
    }
        return root;
}
int main(){
    Node* root = NULL;
    root = insertNode(root,30);
    root = insertNode(root,20);
    root = insertNode(root,10);
    root = insertNode(root,25);
    root = insertNode(root,28);
    cout << "InOrder Traversal of AVL Tree: ";
    InOrder(root);
```

```
    cout << endl;
    return 0;
}
```

```
PS E:\DSA> cd "e:\DSA\Trees\" ; if ($?) { g++
InOrder Traversal of AVL Tree: 10 20 25 28
PS E:\DSA\Trees>
```

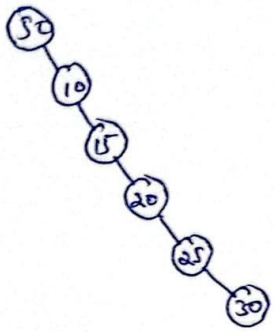
Question#4:

Q NO 4

BST vs AVL

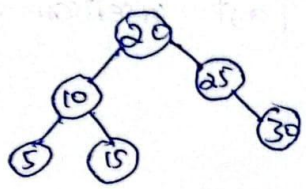
5, 10, 15, 20, 25, 30

BST insertion



H = 6

AVL insertion



H = 3

Comparsion:	BST	AVL
Height	High	Low
Search	O(n)	O(log n)
Worst Case	Bad	Good
Real time	NO	yes

Better Structure:

For Large-Scale File Systems:

AVL Tree prefer as it handle nodes better

For Real-time Application:

AVL-tree perform fast operation.

Conclusion

- AVL balances tree automatically
- provide fast insertion and deletion.